Energy-Efficient and Near Real-Time Routing Algorithm for Disaster Monitoring in Wireless Sensor Networks

Response to the Reviewers — Round 2
April 6, 2026

We sincerely thank the editors and all three reviewers for their continued engagement with our manuscript. We are grateful for the constructive feedback received in this second round of reviews. Below, we address each reviewer's comments with detailed, point-by-point responses. In the accompanying highlighted manuscript, text appearing in **yellow-highlighted boxes** refers to passages that have been added or revised.

The summary of changes undertaken in this revision includes:

Proposed Method (Sec. III)	Two additional algorithms (Graph Construction and Episode Execution) have been added to complement Algorithm 1 (GAPF Training), providing complete pseudocode coverage of the proposed method. A new discussion paragraph comparing GAPF's GCN-based approach with alternative lightweight topology-aware strategies (node-degree heuristics, fuzzy/shallow learning) has been included in the Design Rationale. A comparative analysis of the meta-learning architecture versus end-to-end multi-agent DRL has also been added.
Limitations (Sec. IV)	The discussion on static topology assumptions has been expanded with a concrete analysis of potential step-level fusion and periodic re-evaluation extensions. The homogeneous sensor assumption discussion now addresses how the GCN embedding and CAS mechanism can accommodate heterogeneous sensor capabilities.

Reviewer 1

Reviewer Point 1.1 — The reviewer recommends acceptance and has no additional concerns.

Reply 1.0:

We sincerely thank Reviewer 1 for their thorough evaluation across both rounds of review and for recommending acceptance of our manuscript. Their constructive comments during the first round—particularly regarding training hyperparameters, pseudocode, reward system novelty, deployment feasibility, and execution-time analysis—significantly strengthened the paper. We are grateful for

the time and expertise the reviewer dedicated to improving this work.

Reviewer 2

Reviewer Point 2.1 — The reviewer asks how GAPF's GCN-based approach distinguishes itself from other lightweight, topology-aware strategies—for example, those that leverage node-degree or other local graph features in dynamic link scheduling combined with fuzzy or shallow learning methods.

Reply 2.0:

This is an excellent question that touches on a fundamental design choice in our framework. We provide a detailed comparison below and have added a corresponding discussion to Section III-D (Design Rationale) of the revised manuscript.

1. Representational capacity: learned embeddings vs. hand-crafted features.

Strategies based on node-degree or local graph metrics (e.g., clustering coefficient, betweenness centrality) operate on *pre-defined, single-hop* structural features. While computationally inexpensive, these features capture only the immediate neighborhood of each node and cannot encode higher-order topological patterns such as bottleneck corridors, relay chain potential, or multi-hop connectivity islands.

In contrast, GAPF's two-layer GCN propagates information across the 2-hop neighborhood of every node via the message-passing rule $\mathbf{H}^{(\ell+1)} = \mathrm{ReLU}(\hat{\mathbf{A}}\,\mathbf{H}^{(\ell)}\,\mathbf{W}_\ell)$, then aggregates all node representations into a single graph-level embedding $\mathbf{z} \in \mathbb{R}^{16}$ through mean-pool readout. This embedding implicitly captures:

- **Multi-hop relay potential**: a node with low degree but strategically positioned between two dense clusters is encoded differently from an isolated low-degree node—a distinction invisible to degree-only features.
- **Topology-wide connectivity structure**: the normalised adjacency propagation aggregates the relative spatial distribution of all sensors, encoding whether the network is uniformly distributed, clustered, or exhibits a chain-like structure.
- **Distance-to-BS gradients**: because $\|p_i - p_{\mathrm{BS}}\|$ is an input feature (Eq. 8 in the manuscript), the GCN learns how base-station proximity interacts with local connectivity—e.g., whether sensors near the BS are well-connected relay targets.

2. Adaptability: end-to-end learning vs. manual rule engineering.

Fuzzy inference systems for WSN routing (e.g., Mamdani-type controllers as in QPSOFL) require the designer to manually specify: (i) linguistic variables and membership functions for each input (residual energy, distance, degree), (ii) a rule base mapping input combinations to output actions, and (iii) a defuzzification strategy. This process is topology-specific: rules tuned for a 50-sensor network with 35 m radius do not necessarily transfer to a 100-sensor deployment with 50 m radius.

GAPF avoids this *feature engineering bottleneck* entirely. The GCN weights $\mathbf{W}_1, \mathbf{W}_2$ are learned from data via REINFORCE, automatically adapting to any topology presented during training. Our results across 7 scenarios (20–100 sensors, 25–50 m radius) demonstrate this generalization: GAPF achieves $\geq$98% PDR across all configurations *without per-scenario hyperparameter tuning*, whereas

QPSOFL's fuzzy controller collapses below 15% PDR in sparse topologies where its rules cannot anticipate the connectivity structure.

3. Computational cost comparison.

A legitimate concern is whether the GCN's representational advantage justifies its computational overhead relative to simpler features. We provide a concrete comparison:

Approach	Per-Episode Cost	Parameters	Adapts to New Topology?
Node-degree heuristic	$\mathcal{O}(n^2)$	0	No (fixed rules)
Fuzzy controller	$\mathcal{O}(n \cdot \lvert R\rvert)$	$\lvert R\rvert$ rules	No (requires re-tuning)
Shallow MLP on features	$\mathcal{O}(n \cdot d_f \cdot h)$	$\sim$500	Partially (input-dependent)
GAPF (GCN + CAS)	$\mathcal{O}(n^2 d)$	1,473	**Yes** (learned)

where $|R|$ is the number of fuzzy rules, d_f the hand-crafted feature dimension, h the hidden dimension, and $d = 16$ the GCN embedding dimension. The GCN's $\mathcal{O}(n^2 d)$ cost is incurred *once per episode* (not per step), and with $d = 16$ it amounts to < 0.03 ms on an Apple M-series CPU (Table V in the manuscript). This negligible overhead—smaller than a single routing step—is the cost of gaining topology-wide awareness.

4. Node-degree as input, not as strategy.

It is worth noting that node-degree is already one of the four input features in GAPF's node feature vector (Eq. 8: $\mathbf{x}_i = [\ldots, \deg(i)/\max_j \deg(j)]^\top$). The GCN does not discard local features—it *enriches* them through neighborhood aggregation. In this sense, GAPF subsumes degree-based heuristics as a special case while adding multi-hop structural awareness.

In summary, while node-degree heuristics and fuzzy/shallow methods are valuable for scenarios where computational budgets are extremely tight and topologies are static and known *a priori*, GAPF's GCN encoder provides a strictly richer representation at negligible additional cost, with the critical advantage of automatic adaptation to unseen topologies—a requirement in disaster monitoring where sensor positions are unpredictable.

Manuscript change: A summary of this comparison has been added to Section III-D (Design Rationale) of the revised manuscript.

Reviewer Point 2.2 — The reviewer asks how GAPF's meta-learning configuration with pre-trained frozen experts and episode-level selection differs from end-to-end multi-agent deep reinforcement learning in terms of learning dynamics, memory usage, and per-step computation.

Reply 2.1:

We appreciate this insightful question, which highlights a key architectural decision in GAPF. We provide a systematic comparison along the three dimensions identified by the reviewer.

1. Learning dynamics.

- **End-to-end MARL** (e.g., training a single QMIX or QTRAN from scratch on heterogeneous topologies) must jointly learn: (i) individual agent policies (GRU weights), (ii) value decomposition (mixer weights), and (iii) implicit topology adaptation—all through a single gradient

signal. This creates a non-stationary learning problem: each agent's optimal policy depends on every other agent's simultaneously changing policy. In practice, we observe that standalone QMIX and QTRAN converge to near-identical, mediocre performance (∼55% PDR) because the joint optimisation landscape contains many local optima.

- **GAPF's two-stage approach** separates these concerns: experts are first trained individually on representative topologies until convergence, then frozen. The meta-controller is trained on the *much simpler* problem of mapping topology embeddings to binary expert selection. This 1,473-parameter optimisation problem converges reliably in all seven tested scenarios: REINFORCE with an EMA baseline reaches stable performance within ∼2,000 episodes. The stability arises because the meta-controller's action space is binary and the reward signal (full episodic return) is low-variance compared to per-step TD targets used in end-to-end training.

2. Memory usage.

Component	**End-to-End MARL**	**GAPF (Meta-Learning)**
Agent parameters	$n \times 38{,}983$ (shared)	$n \times 38{,}983$ (shared, frozen)
Mixer parameters	1 mixer (∼50K)	2 mixers (∼100K, frozen)
Meta-controller	—	1,473 (trainable)
Replay buffer	15,000 episodes $\times$ n agents	**None** (on-policy)
Peak training RAM	**3.2–13.8 GB**	**684–951 MB**

The dominant memory cost in end-to-end MARL is the experience replay buffer, which stores full observation–action–reward tuples for all n agents across thousands of episodes. GAPF eliminates this entirely: the meta-controller is trained on-policy (one episode at a time), and expert weights occupy fixed memory. This $13\times$ reduction (Table VII in the manuscript) is what enables GAPF to pass the 1,024 MB feasibility constraint that QMIX/QTRAN fail in every scenario.

3. Per-step computation.

At *execution time*, GAPF's per-step computation is **identical** to standalone QMIX or QTRAN:

$$\mathcal{O}_{\text{step}} = \mathcal{O}(n^2 h + n h^2) \quad \text{(same for all three)}$$

This is because only the selected frozen expert's GRU agents execute at each step—the GCN and CAS controller run *once per episode* (not per step). During training, the overhead of the meta-controller update (one REINFORCE gradient step per episode) is negligible: a single backward pass through 1,473 parameters takes < 0.01 ms.

An end-to-end approach, by contrast, must compute mixer gradients and perform backpropagation through the value decomposition network at every training step, adding $\mathcal{O}(n^2)$ per step for the mixing network.

4. Summary of trade-offs.

Criterion	End-to-End MARL	GAPF	Advantage
Training stability	Low (non-stationary)	High (frozen experts)	GAPF
Sample efficiency	$\sim$200K steps/expert	$\sim$2K episodes (meta only)	GAPF
Peak memory	3.2–13.8 GB	684–951 MB	GAPF ($13\times$)
Per-step latency	$\mathcal{O}(n^2 h)$	$\mathcal{O}(n^2 h)$	**Tied**
Expert diversity	Limited (single policy)	Exploits 2 policies	GAPF
PDR range (7 scenarios)	46–68%	75–99%	GAPF
Flexibility	Any topology, continuous	Episode-level, binary	End-to-End

The main trade-off is that end-to-end methods *could* in principle discover policies that neither QMIX nor QTRAN would find individually, since the search space is unrestricted. However, our empirical evidence shows that the joint optimisation problem is too difficult for current value-decomposition methods to solve reliably across diverse topologies, whereas GAPF's modular design consistently achieves near-optimal performance.

Manuscript change: A concise comparison table and discussion have been added to Section III-D (Design Rationale) of the revised manuscript.

Reviewer Point 2.3 — The reviewer asks how GAPF's episode-level selection design would perform in highly dynamic topologies or rapidly changing channel conditions, and requests a brief discussion on potential extensions to step-level fusion or periodic re-evaluation.

Reply 2.2:

This is a pertinent question that directly relates to the practical deployment of GAPF in real-world disaster scenarios where sensor positions and channel conditions evolve during operation. We provide an analysis of the current design's behaviour and concrete extension paths.

1. Current design: rationale for episode-level selection.

GAPF selects a single expert per episode for two principled reasons:

- **GRU hidden state continuity**: each expert's GRU agents maintain recurrent hidden states $\mathbf{h}_i^{(t)}$ that encode the routing history. Switching experts mid-episode would require re-initialising these hidden states, losing accumulated context about energy levels, packet flows, and relay patterns. This "cold-start" problem degrades performance more than the potential gain from mid-episode adaptation.
- **Temporal scale separation**: in physical WSN deployments, sensor positions change on the order of minutes to hours (even in disaster scenarios with drifting flood sensors or post-earthquake debris displacement), while a routing episode spans seconds. The topology observed at episode onset remains a valid approximation throughout the 100-step episode.

2. Behaviour under topology perturbations within an episode.

Even when the topology changes mid-episode, the selected expert's GRU agents retain partial adaptability through their observations: the local observation $\mathbf{o}_i = [RE_i, CE_i, x_i, y_i, N_i]$ is recomputed at every step, so agents perceive changes in energy levels and (if applicable) positions. The expert selection may become suboptimal, but the agents themselves adapt at the step level. Our results show that even in Scenario 1 (20 sensors, 25 m radius)—where stochastic initial placements

can create near-disconnected topologies—GAPF achieves 75.3% PDR, suggesting robustness to challenging connectivity conditions.

3. Extension paths for highly dynamic scenarios.

We identify three concrete extensions, ordered by implementation complexity:

1. **Periodic re-evaluation** (K-step intervals): Re-run the GCN encoder and CAS controller every K steps (e.g., $K = 20$). The graph $\mathcal{G}$ is reconstructed from current positions, producing an updated embedding $\mathbf{z}_t$. If the CAS controller's selection changes, expert switching occurs with GRU hidden state reset for the newly selected expert. The computational overhead is $\lceil T/K \rceil$ GCN forward passes per episode instead of 1, amounting to $5 \times 0.03\,\text{ms} = 0.15\,\text{ms}$ for $K = 20$—still negligible relative to per-step routing costs.
2. **Confidence-triggered re-evaluation**: Monitor the CAS controller's confidence $|p_{\text{QMIX}} - 0.5|$. When the original selection was marginal (confidence $< \delta$), re-evaluate more frequently. This adaptive scheme concentrates computational overhead on episodes where expert choice is ambiguous—precisely the cases where topology changes are most likely to shift the optimal expert.
3. **Step-level Q-value fusion** (Fusion mode): Instead of selecting a single expert, blend the Q-values from both experts at every step: $Q_{\text{fused}}(s, a) = \alpha_t \cdot Q_{\text{QMIX}}(s, a) + (1 - \alpha_t) \cdot Q_{\text{QTRAN}}(s, a)$, where α_t is a learned mixing weight conditioned on the current graph embedding and hidden states. This eliminates the hidden state incompatibility problem (both experts execute simultaneously) but doubles the per-step computation and requires careful gradient handling to prevent mode collapse.

4. Expected impact.

We anticipate that periodic re-evaluation (option 1) would be sufficient for most disaster-monitoring scenarios, given the temporal scale mismatch between topology changes and episode duration. Step-level fusion (option 3) would be necessary only for extreme scenarios such as drone-assisted WSNs where sensor-relay positions change at sub-second timescales. We emphasise that these extensions are *proposed based on architectural analysis*; their empirical validation constitutes an important direction for future work.

Manuscript change: The Limitations section (item 2, "Static topology assumption") has been expanded with this discussion of extension paths and their computational trade-offs.

Reviewer Point 2.4 — The reviewer asks how GAPF could handle heterogeneous sensor capabilities, and whether the GCN embedding and CAS selection mechanism could naturally incorporate such heterogeneity without significant redesign.

Reply 2.3:

We thank the reviewer for this forward-looking question. We demonstrate below that GAPF's architecture is *structurally compatible* with heterogeneous deployments, requiring only minimal, localised modifications.

1. GCN encoder: extending node features.

The current node feature vector (Eq. 8) is $\mathbf{x}_i \in \mathbb{R}^4$:

$$\mathbf{x}_i = \left[\frac{p_i^{(x)}}{L},\ \frac{p_i^{(y)}}{L},\ \frac{\|p_i - p_{\mathsf{BS}}\|}{\max_j \|p_j - p_{\mathsf{BS}}\|},\ \frac{\deg(i)}{\max_j \deg(j)} \right]^\top .$$

For heterogeneous deployments, this vector can be extended to $\mathbf{x}_i \in \mathbb{R}^{4+k}$ by appending k sensor-specific attributes:

$$\mathbf{x}_i^{\mathsf{het}} = \left[\mathbf{x}_i^\top,\ \underbrace{\frac{E_{0,i}}{E_0^{\max}},\ \frac{P_{\mathsf{tx},i}}{P_{\mathsf{tx}}^{\max}},\ \frac{R_i}{R^{\max}},\ \mathsf{type}_i}_{k \text{ additional features}} \right]^\top$$

where $E_{0,i}$ is the initial energy budget of sensor i, $P_{\mathsf{tx},i}$ its maximum transmission power, R_i its communication radius, and type_i a one-hot or scalar encoding of the sensor hardware class (e.g., temperature, humidity, seismic). The *only* architectural change required is updating the GCN's first-layer weight matrix from $\mathbf{W}_1 \in \mathbb{R}^{4\times d}$ to $\mathbf{W}_1 \in \mathbb{R}^{(4+k)\times d}$, adding $k \times 16 = 16k$ parameters (e.g., 64 parameters for $k = 4$ additional features). The rest of the architecture—GCN propagation, mean-pool readout, CAS controller—remains entirely unchanged.

2. CAS controller: no modification needed.

The CAS controller maps the graph embedding $\mathbf{z} \in \mathbb{R}^{16}$ to a selection probability. Since the GCN readout produces a fixed-dimensional embedding regardless of input feature dimensionality, the CAS controller is agnostic to sensor heterogeneity. It learns to associate topology patterns (which now encode capability distributions) with expert performance, without any structural changes.

3. Per-agent GRU: observation space extension.

Each agent's GRU receives a local observation $\mathbf{o}_i \in \mathbb{R}^5$. For heterogeneous deployments, this can include capability-specific features (e.g., current transmission power level, sensor type indicator), extending $\mathbf{o}_i$ to $\mathbb{R}^{5+m}$. The input projection layer $\mathbf{W}_1 \in \mathbb{R}^{(5+m)\times h}$ absorbs this change with a modest parameter increase of $m \times 64$.

4. Graph construction with heterogeneous radii.

When sensors have different communication radii R_i, the adjacency matrix becomes asymmetric: $A_{ij} = \mathbf{1}[\|p_i - p_j\| \le R_i]$, where an edge from i to j exists if j is within i's range. The GCN handles directed graphs through asymmetric normalisation $\hat{\mathbf{A}} = \tilde{\mathbf{D}}_{\mathsf{out}}^{-1}\tilde{\mathbf{A}}$, which is a well-studied variant requiring no architectural change.

5. Summary of required changes.

Component	Change Required	Parameter Impact
GCN encoder ($\mathbf{W}_1$)	Input dimension $4 \to 4+k$	$+16k$ params
GCN encoder ($\mathbf{W}_2$, readout)	None	0
CAS controller	None	0
Per-agent GRU ($\mathbf{W}_{\text{in}}$)	Observation dim $5 \to 5+m$	$+64m$ params
Graph construction	Asymmetric adjacency	0
Reward scalarization	None	0
Total	**Minimal**	$+16k+64m$

For a realistic heterogeneous deployment with $k = 4$ additional GCN features and $m = 3$ additional observation features, the parameter increase is $16(4) + 64(3) = 256$ parameters—a 17% increase over the current 1,473, well within the computational budget. We note that the analysis above is *architectural*: it demonstrates that GAPF's modular design accommodates heterogeneity without structural redesign. Empirical validation on heterogeneous deployments is identified as a key direction for future work.

Manuscript change: The Limitations section (item 6, "Homogeneous sensor assumption") has been expanded with this analysis in the revised manuscript.

Reviewer 3

Reviewer Point 3.1 — The reviewer notes that they cannot find pseudocode as an algorithm in the proposed method section and requests that pseudocode for the proposed method be added.

Reply 3.0:

We thank Reviewer 3 for this important recommendation. We fully agree that comprehensive pseudocode is essential for reproducibility and clarity, and we recognise that the algorithmic descriptions in the previous revision were insufficient in scope and readability.

Action taken.

In this revision, we have provided **three complete pseudocode descriptions** that together cover the *entire* proposed GAPF method. Each pseudocode listing uses standard procedural conventions—numbered line-by-line steps, explicit loop and conditional constructs, and inline comments explaining each operation—so that a reader can directly translate them into a working implementation.

1. **Algorithm 1: GAPF Training — Pseudocode (CAS Mode)** (Section III-D.3) — Covers the full meta-controller training loop: environment reset, graph construction (calling Algorithm 2), two-layer GCN forward pass, CAS selection probability, stochastic expert sampling, episodic return collection, and REINFORCE parameter update with EMA baseline and gradient clipping. Each step is annotated with an inline comment.
2. **Algorithm 2: Graph Construction — Pseudocode** (Section III-D.2) — Details the construction of the normalised adjacency matrix $\hat{\mathbf{A}}$ and the four-dimensional node feature matrix

$\mathbf{X}$ from raw sensor positions. The pseudocode explicitly shows the pairwise distance check, self-loop augmentation, symmetric normalisation, and per-node feature computation with inline comments for every operation.

3. **Algorithm 3: GAPF Episode Execution — Pseudocode (Inference)** (Section III-D.3) — Describes the full deployment-time procedure in three clearly labelled phases: (Phase 1) one-time topology encoding via GCN, (Phase 2) deterministic expert selection via the CAS controller, and (Phase 3) step-by-step decentralised execution with GRU forward passes, valid-action masking, and reward collection. This pseudocode enables a reader to understand the complete inference pipeline from environment reset to final metric computation.

Complete coverage summary.

Algorithm	Coverage	Section
1	Meta-controller training loop	III-D.3
2	Graph and feature construction	III-D.2
3	Full inference-time execution	III-D.3

Additionally, the reward scalarisation via the Monotonic Q·K Attention Network is fully specified through Equations (3)–(7), the ES meta-learning update rule is given in Section III-C.4, and the energy model is defined in Equations (1)–(2).

Manuscript change: All three algorithms have been added (or substantially rewritten with full inline comments) in Section III-D of the revised manuscript.

End of Round 2 Response to Reviewers